

Relatório Acadêmico de Laboratório

Test Smells e Refatoração em Suítes de Teste

Campo	Identificação
Disciplina	Teste de Software
Trabalho	Laboratório de Test Smells e Refatoração
Aluno	Pedro Augusto Santos Seabra
Matrícula	812784
Data de Entrega	07/10/2026

1. Análise de Smells

Na suíte de testes original, foram identificados os seguintes **Test Smells** que comprometiam diretamente a qualidade, a manutenibilidade e a confiabilidade do código:

- Lógica Condicional e Loops (Teste Inteligente): Encontrado no teste deve desativar usuários se eles não forem administradores. O uso de estruturas for e if/else torna o teste complexo e menos determinístico. O risco associado é que o próprio teste passe a conter bugs, dificultando identificar se uma falha ocorreu na regra de negócio ou na lógica interna do próprio teste.
- Tratamento Incorreto de Exceção (Falso Positivo): Encontrado no teste deve falhar ao criar usuário menor de idade. O teste utiliza um bloco try/catch sem garantir que a exceção seja de fato lançada no bloco try. O risco é que, caso a regra de negócio pare de lançar o erro, o teste passará silenciosamente pelo catch, ocultando uma falha grave de validação do sistema.

- **Teste Desabilitado (Skipped Test):** Encontrado no teste deve retornar uma lista vazia quando não há usuários através do uso do método `test.skip`. O risco é o acúmulo de "código morto" e o mascaramento da cobertura real de testes, criando uma falsa sensação de segurança sobre comportamentos que não estão sendo devidamente validados.

2. Processo de Refatoração

O teste escolhido para demonstrar o processo de refatoração foi o deve desativar usuários se eles não forem administradores, que apresentava excesso de complexidade e violava o princípio de responsabilidade única.

2.1 Código Antes da Refatoração (Código Smelly)

// Código original contendo lógica condicional e loops internos

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum',
    'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com',
    40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

2.2 Código Após a Refatoração (Código Clean)

```
test('deve desativar um usuário comum', () => {
  // Arrange
```

```

    const usuarioComum = userService.createUser('Comum',
'comum@teste.com', 30);
    // Act
    const resultado = userService.deactivateUser(usuarioComum.id);
    const usuarioAtualizado = userService.getUserById(usuarioComum.id);
    // Assert
    expect(resultado).toBe(true);
    expect(usuarioAtualizado.status).toBe('inativo');
  });

test('não deve desativar um administrador', () => {
  // Arrange
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com',
40, true);
  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);
  const usuarioAtualizado = userService.getUserById(usuarioAdmin.id);
  // Assert
  expect(resultado).toBe(false);
  expect(usuarioAtualizado.status).toBe('ativo');
});

```

2.3 Decisões Tomadas

O teste original (*Eager Test*) testava duas regras de negócio distintas em um único bloco utilizando condicionais. Na refatoração, o teste foi dividido em dois testes menores e focados.

Além disso, aplicamos o padrão **AAA (Arrange, Act, Assert)** de forma rigorosa, eliminando totalmente as estruturas de controle (`for` e `if/else`) para garantir que o fluxo de execução do teste seja linear, descritivo, determinístico e direto ao ponto.

3. Relatório da Ferramenta

```
PS C:\Users\pedro\Desktop\testSmelly\test-smelly> npx eslint .

C:\Users\pedro\Desktop\testSmelly\test-smelly\test\userService.smelly.test.js
 44:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 46:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 49:9  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 73:7  error  Avoid calling `expect` conditionally`  jest/no-conditional-expect
 77:3  warning Tests should not be skipped          jest/no-disabled-tests
 77:3  warning Test has no assertions          jest/expect-expect

X 6 problems (4 errors, 2 warnings)

PS C:\Users\pedro\Desktop\testSmelly\test-smelly> 
```

A ferramenta de análise estática **ESLint**, configurada com o plugin do **Jest**, foi essencial para automatizar a detecção de vulnerabilidades e maus padrões na suíte.

Conforme demonstrado na imagem do relatório acima:

- * O linter apontou **6 problemas no total**, cruzando perfeitamente com a análise manual realizada previamente.
- * Indicou as violações da regra `jest/no-conditional-expect` nos `ifs` estruturados erroneamente.
- * Advertiu sobre o teste ignorado por meio da regra `jest/no-disabled-tests`.

A refatoração realizada eliminou **100%** desses avisos e erros apontados pela ferramenta.

4. Conclusão

A escrita de testes limpos (**Clean Tests**), alinhada ao uso contínuo de ferramentas de análise estática como o ESLint, é indispensável para a sustentabilidade de qualquer software.

Test Smells criam uma falsa sensação de segurança, pois testes frágeis ou mascarados podem aprovar implementações defeituosas sem alertar a equipe de desenvolvimento. Ao aplicar boas práticas como o padrão **AAA** e a separação de responsabilidades, garantimos que a suíte atue como uma

verdadeira rede de segurança e uma documentação viva do sistema, reduzindo custos de manutenção e facilitando o trabalho da equipe a longo prazo.